

100 RAG Interview Questions - With Detailed Answers

The Complete Study Guide for AI Engineers · 2026 Edition

Who this is for: Software engineers and students prepping for AI Engineer interviews where RAG comes up - which is roughly 36% of all AI Engineer roles. RAG is *the* #1 most-asked AI engineering topic in 2026.

How to use this doc: Don't memorize. Try to answer each question out loud in under 60 seconds, then read the answer to see what you missed. If you can confidently answer 70+ out of 100, you're ready for senior RAG interviews.

Category 1 - RAG Fundamentals (Questions 1–20)

1. What is RAG and what problems does it actually solve?

RAG (Retrieval-Augmented Generation) is a pattern where an LLM is given relevant context retrieved from an external knowledge base *at inference time*, instead of relying only on what it learned during training. It solves three core problems: knowledge cutoffs (LLMs don't know recent or private data), hallucinations (the model invents facts when ungrounded), and verifiability (you can't cite the source of a pure LLM output). RAG essentially turns a generic LLM into a domain-aware system without retraining.

2. Walk me through the components of a RAG pipeline end-to-end.

A RAG pipeline has two phases. **Ingestion (offline):** documents are loaded, chunked into manageable pieces, embedded into vector representations, and stored in a vector database with metadata. **Inference (online):** the user query is embedded, the top-k most similar chunks are retrieved (often reranked), the retrieved chunks are injected into the prompt as context, and the LLM generates an answer grounded in that context. Production systems usually add query rewriting, hybrid search, reranking, citation enforcement, and evaluation hooks on top of this base flow.

3. When would you choose RAG over fine-tuning?

Choose RAG when your data changes frequently, when you need verifiable sources, when you can't risk training-data leakage, or when your corpus is too large to fit in a fine-tuning dataset. Choose fine-tuning when you need to teach the model a *style* or *behavior* (tone, format, refusal patterns), or when latency/cost demands a smaller model that's been specialized. In practice, most production systems use RAG for *what* the model says (facts) and fine-tuning for *how* it says it (style) - they're complements, not alternatives.

4. RAG vs longer context windows - when does which win?

Long context wins for small, fixed corpora that fit entirely in the window (e.g., a single 200-page contract). RAG wins for large corpora (millions of documents), frequently changing data, and cost-sensitive use cases - even with cheap long-context models, dumping irrelevant context wastes tokens and degrades quality due to "lost-in-the-middle" effects. The sweet spot is often hybrid: use RAG to narrow down to the top 20 chunks, then let a long-context model reason over them.

5. What are the failure modes of a basic RAG system?

Top failure modes: (1) retrieval misses - the right chunk wasn't in the top-k; (2) chunking too small or too large - losing context or burying signal; (3) the LLM ignores retrieved context and falls back to parametric knowledge; (4) the LLM hallucinates *grounded* answers - wrong facts justified by misleading context; (5) query-document vocabulary mismatch - the user asks in different words than the document uses; (6) outdated index - the source updated but the embeddings didn't. Every production RAG system needs telemetry on each of these.

6. Explain the difference between extractive and generative QA.

Extractive QA pulls the exact answer span verbatim from a source document (like Google's "featured snippets") - high precision, no hallucination risk, but rigid. Generative QA uses an LLM to synthesize an answer in natural language using retrieved context - more flexible, handles multi-document reasoning, but can hallucinate. Modern RAG is generative QA with constraints (cite sources, refuse if context insufficient) to get the flexibility of generation with the safety of extraction.

7. When is RAG the wrong tool for the job?

RAG is wrong when: (1) the question doesn't require external knowledge (e.g., "translate this to French"); (2) the data is highly structured and a SQL query would do better; (3) you need precise computation or aggregation (use tools/agents instead); (4) the corpus is small enough to fit in context (just stuff it in); (5) latency requirements are sub-100ms and retrieval adds too much; (6) the use case requires reasoning *across* the entire corpus rather than from a few documents. Don't reach for RAG just because it's trendy.

8. What is the role of embeddings in RAG?

Embeddings are dense vector representations of text that capture semantic meaning - texts with similar meaning map to nearby vectors in high-dimensional space. In RAG, both documents and queries are embedded into the same space, and retrieval is just "find the document vectors closest to my query vector" (usually by cosine similarity). The quality of your embedding model is one of the biggest determinants of retrieval quality - a bad embedding model can't be saved by a better LLM downstream.

9. What is the difference between RAG and search?

Traditional search returns *documents* and lets the human read them. RAG retrieves chunks and then uses an LLM to synthesize a natural-language answer grounded in those chunks. Search is optimized for relevance ranking; RAG additionally needs faithfulness, citation, and synthesis. A good way to think about it: search ends where RAG begins.

10. What's the difference between a retriever and a reranker?

A retriever does fast first-pass filtering - usually vector similarity over the entire corpus, returning the top-k (say, k=50) candidates. A reranker takes those candidates and scores them more precisely using a heavier model (often a cross-encoder), reordering them and returning the top-N (say, N=5). Retrievers optimize recall (don't miss the right answer); rerankers optimize precision (put the right answer first).

11. Can RAG work without an LLM?

Sort of - pure retrieval with extractive QA (like older search engines or QA systems with BERT-style readers) achieves the retrieval half of RAG without generation. But "RAG" as a term specifically implies *Generation* - the synthesis step where an LLM combines retrieved context into a natural answer. Without that, you have search or extractive QA, not RAG.

12. What does "grounding" mean in RAG?

Grounding means tying the LLM's response to specific, verifiable source material - the retrieved chunks. A grounded answer can be traced back to a source ("this claim came from document X, paragraph 3"). Ungrounded answers come from the model's parametric memory and have no verifiable source. Strong grounding requires both providing relevant context *and* enforcing - via prompt engineering or output validation - that the model uses that context.

13. What is "context stuffing" and why is it a problem?

Context stuffing means cramming as much retrieved content into the prompt as possible, hoping the LLM finds the relevant parts. Problems: (1) cost scales linearly with context tokens; (2) latency goes up; (3) LLMs suffer "lost-in-the-middle" - they pay more attention to the start and end of context than the middle; (4) irrelevant context can actually *degrade* answer quality by distracting the model. Better approach: retrieve more, then rerank tightly, and pass only the top 3–5 chunks.

14. What's the "lost in the middle" problem?

Research (Liu et al., 2023, and replicated since) shows LLMs pay disproportionate attention to the beginning and end of their context window, with accuracy degrading sharply for information placed in the middle. Practical implication for RAG: ordering matters. Put the most relevant chunks at the start or end, not buried in the middle. Some systems explicitly re-order retrieved chunks so the highest-ranked piece is first, the second-highest is last, and lower-ranked chunks fill the middle.

15. What is a "chunk" in RAG?

A chunk is a unit of text - typically 200–1000 tokens - that gets independently embedded and indexed. Documents are too large to embed as a whole (and similarity over a whole document is meaningless), so they're split into chunks. The chunk is the atomic unit of retrieval: when the retriever returns the top-k, it's returning k chunks, not k documents. Chunk boundary choices have outsized impact on retrieval quality.

16. What metadata should you store with each chunk?

At minimum: document ID, source URL or file path, chunk index within the document, character or token offsets, ingestion timestamp, and any access-control tags (user_id, team_id, role). Beyond minimum: document title, section/heading, author, publication date, language, document type (PDF, HTML, etc.), and any domain-specific tags (product line, geography, customer tier) that you might want to filter on at query time.

17. What's the difference between sync and async RAG?

Sync RAG: user query → retrieve → generate → return. Everything blocks the user. Async/streaming RAG: retrieval still blocks, but the generation streams tokens to the user as they're produced. Some advanced systems also do *speculative retrieval* - starting retrieval before the user finishes typing (predicting from partial queries). In production, you almost always want streaming for the generation step to keep perceived latency low.

18. What's a "context window budget" and how do you manage it?

The context window budget is the total tokens the LLM can process per request. You spend it across system prompt, conversation history, retrieved context, and the model's output. Budgeting means deciding how many tokens to allocate to each - e.g., 500 for system prompt, 2000 for history, 4000 for retrieved chunks, leaving 2000 for generation. When the budget tightens, you have to choose what to drop: shorter history, fewer chunks, smaller chunks, or output truncation.

19. What is the difference between recall and precision in RAG retrieval?

Recall = of all the chunks that *should* have been retrieved (the truly relevant ones), how many did we actually return? Missing the right chunk is a recall failure. **Precision** = of the chunks we did return, what fraction were actually relevant? Returning lots of garbage is a precision failure. In RAG you want both, but they trade off - wider retrieval (higher k) improves recall but hurts precision. Reranking is the typical fix: cast a wide net (recall), then filter tight (precision).

20. What's a "naive RAG" and how does it differ from production RAG?

Naive RAG = chunk documents → embed → store → embed query → top-k similarity → stuff in prompt → generate. Production RAG adds: query understanding (rewriting, decomposition, expansion), hybrid retrieval (dense + sparse), reranking, metadata filtering, multi-step retrieval for complex queries, citation enforcement, evals, caching, fallbacks, observability, and version control on prompts. The gap between naive and production is where most engineering effort lives.

Category 2 - Chunking & Embeddings (Questions 21–40)

21. How do you pick the right chunk size for your use case?

Start with 500 tokens with 50-token overlap as a default. Adjust based on: document structure (Q&A pairs → small chunks; legal contracts → larger chunks); embedding model context length; how specific vs broad your queries tend to be (specific → smaller; broad → larger); and downstream LLM context budget. The honest answer in an interview: "I'd start at 500/50, evaluate retrieval quality on a golden set, and tune from there." Hand-waving isn't acceptable - you'd evaluate empirically.

22. What chunking strategies do you know? When to use which?

(1) **Fixed-size**: every N tokens, simple, fast, but cuts mid-sentence - fine for baselines. (2) **Recursive**: split on paragraph, then sentence, then word, respecting structure - good general default. (3) **Semantic**: split where embedding similarity drops between adjacent sentences - best quality, more expensive. (4) **Document-aware**: split on markdown headers, HTML tags, code-block boundaries - best for structured docs. (5) **Parent-child / hierarchical**: small chunks for retrieval, larger parent chunks for generation context - best of both worlds.

23. What is semantic chunking and when is it worth the cost?

Semantic chunking computes embeddings for each sentence and creates a new chunk boundary wherever the cosine similarity between adjacent sentences drops below a threshold (i.e., the topic is shifting). Worth the cost when document structure is loose (no clean headings/paragraphs), when queries are very specific, or when retrieval precision is critical (medical, legal). Not worth it for well-structured documents where document-aware splitting already aligns with topic boundaries.

24. What is chunk overlap and why do you need it?

Chunk overlap means the last N tokens of chunk i are also the first N tokens of chunk i+1. You need it because relevant information can fall on a chunk boundary, and without overlap, both chunks miss it. Typical overlap is 10–20% of chunk size. Trade-off: more overlap = more storage and more retrieved duplicates to deduplicate. Sweet spot is usually 50–100 tokens for 500-token chunks.

25. How do you handle tables in chunking?

Don't split tables mid-row. Options: (1) keep the entire table as one chunk regardless of size; (2) convert table to markdown or a structured text format (CSV-like, sentence-per-row) before embedding; (3) treat each table row as its own chunk with the header repeated; (4) store the table as metadata alongside its chunk and use the surrounding text for retrieval. The right approach depends on whether queries reference table contents or surrounding narrative.

26. How do you handle code in chunking?

Don't split mid-function. Use language-aware splitters (e.g., LangChain's `RecursiveCharacterTextSplitter` with language-specific separators or AST-based

splitters like tree-sitter). Preserve function/class boundaries. Keep imports with the code they apply to. For RAG over codebases, also consider embedding function signatures and docstrings separately from full implementations - they're often what users actually search for.

27. How do you handle images and figures in chunking?

Three options: (1) use a multimodal embedding model that can embed images directly (CLIP, OpenAI's text-embedding-3 with image, etc.); (2) run image captioning first (e.g., GPT-4V) and treat the caption as the chunk's content; (3) ignore images but keep their captions and surrounding text. Production systems often do (2) - generate rich captions during ingestion and index those alongside the original image reference for display.

28. How do you pick an embedding model for production?

Evaluate on: (1) **MTEB leaderboard performance** on your task type (retrieval, semantic similarity, classification); (2) **domain match** - general-purpose models (text-embedding-3, BGE) vs. domain-specific (BioBERT for medical, FinBERT for finance); (3) **dimensions** - higher dimensions = better quality but more storage/compute; (4) **cost and latency** - hosted (OpenAI, Cohere) vs. self-hosted (BGE, e5); (5) **multilingual needs**. Always benchmark 2-3 candidates on your own retrieval eval set before committing.

29. When would you train or fine-tune your own embeddings?

When off-the-shelf embeddings systematically miss your domain's semantics - e.g., medical jargon where "MI" means myocardial infarction, or financial terms where "short" is a verb. Fine-tune when: (1) you have at least a few thousand labeled query-document pairs; (2) general-purpose embeddings underperform on your eval set despite tuning everything else; (3) the cost/effort of training is justified by the gain. Often, hybrid search + reranking gets you 80% of the gain without training.

30. What's the difference between bi-encoders and cross-encoders?

Bi-encoders embed query and document independently and compare via similarity (cosine, dot product). Fast - pre-compute document embeddings once, only the query needs embedding at query time. Used for retrieval. **Cross-encoders** take query and document *together* as input and output a relevance score directly. More accurate (the model can attend across both) but slow - can't pre-compute. Used for reranking. Standard architecture: bi-encoder retrieves 50, cross-encoder reranks to 5.

31. What is the "parent-document retrieval" pattern?

Index small chunks for precise retrieval, but when a chunk is retrieved, fetch and pass its larger *parent* (e.g., the full section or paragraph it came from) to the LLM. This gives you the retrieval precision of small chunks with the contextual completeness of large ones. Implementation: store both small and large chunks, with the `parent_id` on each small chunk, and join at query time.

32. What are sparse embeddings (BM25, SPLADE)?

Sparse embeddings represent text as a high-dimensional vector where most values are zero, with non-zero values for the terms in the text (and sometimes related terms). **BM25** is the classical example - pure keyword matching with term-frequency / inverse-document-frequency weighting. **SPLADE** is a neural sparse model that learns to expand and weight terms. Sparse retrieval is strong on exact keyword matches, especially for rare terms or named entities where dense models can blur the signal.

33. What does dimensionality mean for embeddings?

Embedding dimensions = the length of the vector representing each text (e.g., 384, 768, 1536, 3072). Higher dimensions generally capture more nuance but cost more in storage (linear) and computation (matters for indexing and similarity calculations at scale). Practical guidance: 384–768 is enough for most apps; 1536 is overkill unless you need maximum quality; 3072 is for cases where every percentage point of recall matters.

34. What is Matryoshka Representation Learning (MRL)?

MRL is a training technique where embeddings are trained so that *prefixes* of the full vector are themselves valid embeddings of lower dimensionality. So a 3072-dim Matryoshka embedding can be truncated to 1536 or 768 dims and still work, just with slightly less accuracy. Practical use: store full embeddings, but use truncated versions for fast first-pass retrieval, then re-score with full embeddings on the top candidates. OpenAI's text-embedding-3 uses MRL.

35. What's the difference between sentence embeddings and word embeddings?

Word embeddings (Word2Vec, GloVe) give one vector per *word*, ignoring context - "bank" has the same embedding whether it means a financial institution or a river bank. Sentence embeddings (Sentence-BERT, modern embedding models) give one vector per *sentence or chunk* using context-aware transformer models, so "bank" gets different embeddings in different sentences. RAG uses sentence-level embeddings exclusively - word embeddings are too coarse for retrieval.

36. How do you handle multilingual RAG?

Three approaches: (1) **Multilingual embedding model** (e.g., multilingual-e5, BGE-M3) - embeds all languages into a shared space, so a query in English can retrieve documents in Hindi. (2) **Translate to a pivot language** (usually English) at ingestion and at query time - simple but loses nuance. (3) **Per-language indexes** - separate index per language, plus language detection at query time. (1) is the modern default. Always evaluate on your actual language mix because quality varies a lot across languages.

37. What is "instruction-tuned" embedding?

Some embedding models (e.g., Instructor, text-embedding-3, BGE) accept an instruction prefix like "Represent this sentence for retrieving similar documents:" before embedding. The same text gets different embeddings depending on the task instruction. Use cases: same text might need different embeddings for retrieval vs. classification vs. clustering. In

RAG, you typically use an instruction for queries (something like "Find passages that answer this question:") and a different one for documents.

38. How do you embed long documents?

You don't - embedding models have token limits (256–8192 typically), and beyond that, the embedding becomes meaningless (an average of too many ideas). You chunk first, then embed each chunk. If you want a "document-level" representation for filtering or routing, embed the title plus the first 500 tokens, or use a hierarchical embedding approach (embed chunks, average them, optionally rerun on the averages).

39. What's a typical embedding latency budget?

For a hosted API like OpenAI or Cohere: 50–200ms per query embedding, with batch endpoints letting you embed thousands of documents per second at lower cost. For self-hosted models on GPU: under 50ms per query, thousands per second for batch ingestion. Production rule of thumb: query embedding should not be your bottleneck - if it is, you're probably calling the API one query at a time when you could batch, or you're using a hosted model when self-hosted is fast enough.

40. What is embedding drift and how do you detect it?

Embedding drift = the distribution of embeddings produced by your model shifts over time, typically because (1) the embedding model was updated by the vendor or (2) new types of documents are entering the corpus that the model wasn't designed for. Detect by tracking: average pairwise similarity across new docs, retrieval quality on a fixed eval set over time, and the distribution of similarity scores at query time. Fix by re-embedding the corpus (expensive but sometimes necessary) or pinning a model version.

Category 3 - Vector DBs & Retrieval (Questions 41–60)

41. Compare pgvector, Pinecone, Weaviate, Qdrant. When does which win?

pgvector: Postgres extension. Free, you already have Postgres, great for under ~10M vectors and when you want transactional semantics alongside vector search. **Pinecone**: hosted, very fast, hands-off, but pay-per-vector pricing gets expensive at scale. **Weaviate**: open-source, strong hybrid search, good for complex filtering. **Qdrant**: open-source, very fast, written in Rust, best self-hosted performance per dollar. Decision tree: starting out → pgvector; need managed and don't care about cost → Pinecone; need open-source with hybrid → Weaviate; need self-hosted performance → Qdrant.

42. What is HNSW and why is it the default index for vector DBs?

HNSW = Hierarchical Navigable Small World. It's an approximate nearest neighbor (ANN) index that builds a multi-layer graph where each node connects to its nearest neighbors, with sparser layers on top for fast navigation. Query: start at a high layer, greedily walk toward the query, descend layers, refine. Why default: excellent recall (95%+) at very low latency

(sub-10ms for millions of vectors), tunable via two parameters (M and ef), and no need for retraining when adding new vectors.

43. What's the difference between exact and approximate nearest neighbor search?

Exact (brute force): compute similarity between query and every vector. $O(N)$ per query - fine up to ~100K vectors, infeasible at millions. **Approximate (ANN):** use an index (HNSW, IVF, ScaNN) that trades a tiny bit of accuracy for huge speed gains - millisecond queries at billions of vectors. In production RAG, ANN is always the default; exact search is used only for evaluation (to compute "ground truth" recall of the ANN index).

44. Dense vs sparse retrieval - when do you need both?

Dense (embeddings) is strong on semantic similarity, paraphrases, and concepts - "what causes high blood pressure" matches "hypertension etiology." **Sparse** (BM25) is strong on exact matches, rare terms, named entities, product codes, error messages - "ERR-4017" needs to match exactly. You need both when your queries mix conceptual questions with specific identifiers. Most production systems use hybrid search; pure dense fails on rare-term queries, pure sparse fails on rephrased queries.

45. What is hybrid search and how would you implement it?

Hybrid search combines dense (semantic) and sparse (lexical, e.g., BM25) retrieval. Implementation options: (1) **Run both, then merge** using Reciprocal Rank Fusion (RRF) - robust and doesn't require score normalization. (2) **Weighted score combination** - needs careful normalization since BM25 and cosine scores are on different scales. (3) **Use a vector DB with native hybrid** (Weaviate, Qdrant, Elasticsearch with vector support). RRF is the most common production choice because it's parameter-free and works well out of the box.

46. What is Reciprocal Rank Fusion (RRF)?

RRF combines multiple ranked lists into one. For each result, compute $1 / (k + \text{rank})$ from each list (k is a constant, typically 60), then sum across lists. Higher final score = higher rank. Why it works: it's robust to score scale differences (only ranks matter, not raw scores), simple to implement, parameter-light (just k), and consistently performs well across domains. Standard tool in any hybrid retrieval setup.

47. How does reranking work and when is it worth the latency?

Rerankers (typically cross-encoders like Cohere Rerank or BGE-reranker) take the top-k retrieved candidates (say k=50) and the query, and score each candidate's relevance more precisely than embedding similarity. They reorder, returning the top-N (say N=5). Worth the latency (50–200ms) when: precision matters more than minor latency increase; you're seeing relevant-but-not-best chunks ranked high; or the corpus is large enough that retrieval alone misses nuances. Almost always worth it for production.

48. How do you implement metadata filtering at scale?

Two patterns: (1) **Pre-filter**: filter candidates by metadata first, then run vector search on the filtered set - simple but expensive if filters are unselective. (2) **Post-filter**: run vector search, then filter results by metadata - fast but may not return enough results. (3) **Hybrid (filter-aware ANN)**: vector DBs like Pinecone, Qdrant, and Weaviate maintain filter-aware indexes that integrate filtering into the vector search itself - best of both worlds. Always benchmark with your actual filter selectivity.

49. How do you handle multi-tenant data isolation in a vector DB?

Three options: (1) **One namespace/collection per tenant** - strong isolation, but expensive at thousands of tenants. (2) **Metadata-based isolation** in a shared index - every chunk tagged with `tenant_id`, every query filtered by it. Cheaper but riskier (one bug = data leak). (3) **Tier isolation** - top customers get their own namespace, small customers share. The "shared with metadata filter" approach is most common, with rigorous testing of the filter logic.

50. What's the difference between cosine similarity, dot product, and Euclidean distance?

Cosine = dot product of normalized vectors; measures angle, ignores magnitude. Standard for embeddings since most embedding models are trained for it. **Dot product** = cosine times magnitude; faster (no normalization needed) but assumes embeddings are already normalized or that magnitude carries signal. **Euclidean (L2)** = straight-line distance in vector space. For most modern embeddings (which are L2-normalized at training), cosine and dot product give identical rankings. Pick whichever your vector DB optimizes for.

51. How do you handle large-scale ingestion (millions of documents)?

(1) **Batch embedding**: don't call the embedding API one document at a time - batch hundreds per request. (2) **Parallelize** across workers (Celery, Ray, or just multiprocessing). (3) **Use bulk-insert APIs** of the vector DB - every DB has them and they're 10–100x faster than single inserts. (4) **Stage embeddings**: write to S3/Parquet first, then bulk-load - gives you retry-ability if vector DB ingestion fails. (5) **Pre-compute on cheaper hardware** offline rather than during a deploy.

52. What is "vector quantization" and when do you use it?

Vector quantization compresses each embedding from float32 (4 bytes per dimension) to lower-precision formats: float16 (2x compression), int8 (4x compression), or binary (32x compression). Trade-off: minor recall loss for massive storage and speed gains. Use it when: corpus is 10M+ vectors, memory/storage is a bottleneck, or you need sub-millisecond queries. **Product Quantization (PQ)** and **Scalar Quantization** are the two common families. Most vector DBs support this natively.

53. What's the difference between IVF, HNSW, and ScaNN indexes?

IVF (Inverted File): clusters vectors into N partitions, searches only the closest M partitions at query time. Fast, simple, but recall depends heavily on cluster quality. **HNSW (Hierarchical Navigable Small World)**: graph-based, very high recall, slightly more

memory. Default choice. **ScaNN** (Google): uses anisotropic quantization, excellent on high-dim embeddings, used inside Google. Most production systems use HNSW unless they're operating at Google scale.

54. How do you handle real-time updates to a vector index?

Vector DBs handle this differently: **Pinecone**, **Qdrant**, **Weaviate** support live upsert/delete with eventual consistency. **pgvector** with **ivfflat** index degrades as you insert without rebuilding; **hnsw** index handles inserts well. Key pattern: separate "hot" (recently added, smaller, frequently updated) from "cold" (bulk, rarely changed) indexes, query both, merge results. For high-write workloads, batch inserts every few seconds rather than per-document.

55. What's a "namespace" or "collection" in a vector DB?

A logical container that separates one set of vectors from another, even within the same vector DB instance. Used for: multi-tenancy (one namespace per customer), versioning (current vs. previous index for A/B testing), or topic separation (one namespace per knowledge area). Queries are scoped to a single namespace by default. Think of namespaces like database schemas - same engine, separate data.

56. How do you do "search-as-you-type" with vector search?

Standard vector search isn't great for incremental queries because each character changes the embedding. Solutions: (1) **Debounce** - only embed and search every 300ms while typing. (2) **Hybrid with autocomplete** - use traditional prefix search (trie or Elasticsearch) for autocomplete, dense search only when the user pauses. (3) **Cache common partial queries**. (4) **Speculative search** - embed partial queries and pre-fetch. Most production search-as-you-type stays on traditional autocomplete and reserves vector search for finalized queries.

57. What is "cold-start" in vector search and how do you handle it?

Cold start = serving queries before the index is fully built (or warmed in memory). For HNSW, warming means loading the index into RAM. For ANN indexes, the first few queries on a fresh process can be 10x slower as caches fill. Mitigations: (1) preload index at deploy; (2) run synthetic queries at startup to warm caches; (3) keep indexes in shared memory across worker restarts; (4) for serverless functions, use provisioned concurrency.

58. How does GPU help with vector search?

GPUs dramatically speed up brute-force similarity and ANN construction (batched matrix multiplies are GPU-native). FAISS-GPU can do millions of vector comparisons in milliseconds. Use GPUs when: extremely large indexes (>100M vectors), high QPS where CPU is bottleneck, or training/rebuilding indexes regularly. Trade-off: GPU cost is high - most systems under 100M vectors run fine on CPU with HNSW.

59. What is "query expansion" in retrieval?

Query expansion enriches the original query with related terms before retrieval to improve recall. Approaches: (1) **Synonyms/thesaurus**: classical, brittle. (2) **LLM-based**: ask an LLM to rephrase or add related terms - most common modern approach. (3) **Pseudo-relevance feedback**: do an initial retrieval, take top results, extract terms, re-query. Trade-off: better recall, but can introduce noise and increase latency. Useful for short or vague queries.

60. What is "query decomposition" in retrieval?

Splitting a complex query into sub-questions, retrieving for each, and combining results. Example: "Compare the Q3 revenue of Apple and Microsoft" → ["Apple Q3 revenue", "Microsoft Q3 revenue"]. Typically done by an LLM. Used when single-step retrieval is unlikely to find all relevant context (multi-hop questions). Adds latency but dramatically improves answer quality on complex queries. Standard in research RAG systems (RAPTOR, ReAct-style retrievers).

Category 4 - Evaluation (Questions 61–80)

61. How do you evaluate a RAG system end-to-end?

Two layers: **Retrieval evaluation** (did we get the right context?) using metrics like precision@k, recall@k, MRR, and NDCG against a labeled set. **Generation evaluation** (did we answer well given that context?) using faithfulness (is the answer supported by retrieved context?), answer relevance (does it answer the question?), and answer correctness (is it actually right?). Production also adds end-to-end metrics: user satisfaction, click-through on citations, escalation rate. Frameworks like RAGAS and TruLens automate much of this.

62. What metrics measure retrieval quality?

Precision@k: of the top-k retrieved chunks, how many are relevant? **Recall@k**: of all relevant chunks, how many appear in top-k? **MRR (Mean Reciprocal Rank)**: average of $1/(\text{rank of first relevant result})$ - penalizes burying the right answer. **NDCG (Normalized Discounted Cumulative Gain)**: weighs rank position with logarithmic discount; the gold standard when you have graded relevance (not just binary). For production RAG, recall@10 and MRR are the most commonly tracked.

63. What metrics measure generation quality?

Faithfulness: does the answer only make claims supported by retrieved context? (LLM-as-judge usually). **Answer relevance**: does the answer actually address the question? **Answer correctness**: is the answer factually right, judged against a ground-truth answer? **Context precision**: of the retrieved chunks, how many were actually used in the answer? **Context recall**: was all the necessary info in the retrieved chunks? Faithfulness is the most important - an unfaithful answer is a hallucination, full stop.

64. How do you build a golden dataset for RAG?

(1) **Collect real queries** - from production logs, user research, or synthetic generation. (2) **For each query, identify the relevant chunks** - human annotators or domain experts. (3) **Write or extract ideal answers**. (4) **Tag failure modes** - hallucination, missing context, irrelevant context, etc., so you can break down failures. Aim for 100–300 high-quality examples to start. Quality beats quantity. Synthetic generation (LLMs creating Q&A pairs from your corpus) is great for bootstrapping but should be human-validated.

65. What is LLM-as-judge for RAG and what are its limits?

LLM-as-judge: use a powerful LLM (GPT-4, Claude) to score outputs of your RAG system against criteria (faithfulness, relevance, etc.). Limits: (1) **Bias** - judges prefer outputs from their own model family; (2) **Inconsistency** - the same judge can give different scores on different runs; (3) **Cost** - expensive at scale; (4) **Reasoning errors** - judges can be wrong about subtle factual claims; (5) **Position bias** - when comparing two answers, judges favor the first one shown. Mitigations: rotate models, use chain-of-thought prompts, randomize order, calibrate against human scores.

66. How do you detect retrieval drift in production?

Track over time: (1) **Similarity score distributions** - if the average similarity between query and top result drops, queries are getting harder or the index is decaying. (2) **Query-document overlap** - are queries hitting different documents than before? (3) **No-result rate** - fraction of queries where top similarity falls below a threshold. (4) **Click-through on citations** - drops mean users aren't finding answers. (5) **Periodic eval on a fixed golden set** - direct measure. Alert when any of these moves >2 std dev from baseline.

67. How do you regression-test RAG in CI/CD?

(1) Maintain a golden set of 100+ Q&A pairs in your repo. (2) On every PR that changes the RAG pipeline (prompts, chunking, retrieval params), run the golden set and compute eval metrics. (3) Compare to the main branch's metrics - fail the build if any metric regresses by more than X%. (4) Cache embeddings of the golden set to keep CI fast. (5) Log per-question diffs so reviewers can see exactly which questions got better or worse. This is standard but most teams skip it; it's the single highest-leverage practice for reliability.

68. What is "context precision" and how does it differ from precision@k?

Precision@k is purely retrieval: of the top-k retrieved chunks, how many are labeled as relevant in your eval set? **Context precision** (RAGAS terminology) is generation-aware: of the retrieved chunks, how many were *actually used* to generate the answer? Measured by LLM-as-judge looking at the answer and asking "which retrieved chunks were necessary?" Context precision can be high even if precision@k is low - the model can ignore irrelevant retrieved chunks. But low context precision plus high precision@k means the model is hallucinating despite good retrieval.

69. What is "context recall" and why does it matter?

Context recall: was *all* the information needed to answer the question actually present in the retrieved chunks? If context recall is low, even a perfect LLM can't give a complete answer - the info just wasn't there. Measured by comparing the ground-truth answer to retrieved chunks and asking (via LLM-as-judge) whether each claim in the ground truth is supported by retrieved context. Low context recall = retrieval problem, not generation problem.

70. How do you measure faithfulness?

Run the RAG system to get an answer and the retrieved context. Then ask an LLM judge: "For each claim in this answer, is it supported by the provided context?" The faithfulness score is the fraction of claims that are supported. Implementation note: extracting "claims" from an answer is itself an LLM step - typically a separate prompt that breaks the answer into atomic factual statements. RAGAS does this automatically. A faithfulness score below 0.85 is usually a red flag.

71. What's the difference between online and offline evaluation?

Offline eval: run on a fixed golden dataset, in CI or before deployment. Fast, deterministic, but limited - covers only what's in the dataset. **Online eval:** measure metrics on real production traffic. Includes user feedback (thumbs up/down, edits, escalations), implicit signals (time spent, clicks on citations), and A/B tests of new versions. Online catches issues offline misses (long-tail queries, distribution shift) but is slower and noisier. Production RAG systems need both.

72. How do you A/B test RAG changes safely?

(1) **Shadow traffic:** run new version on a copy of production traffic, compare outputs without showing to users. Zero risk, slow signal. (2) **Canary:** route 1% of traffic to new version, monitor key metrics, ramp up if healthy. (3) **Interleaving:** for query, show outputs from both versions side-by-side, measure which gets clicked/preferred. Most expressive but UI-dependent. Always log enough metadata to attribute outcomes to versions. Always have an automated rollback trigger on key metric regressions.

73. How do you evaluate "I don't know" responses?

Critical: a RAG system should refuse to answer when retrieved context is insufficient, but evaluating *when* to refuse is hard. Build an explicit "abstain set" in your golden data: questions that the system *should* refuse. Score on both (1) **refusal accuracy** - did it refuse questions it should have refused, and answer ones it should have answered? - and (2) **answered question quality** - for the questions it did answer, are they good? Penalize models that refuse too often (overcautious) or too rarely (hallucination risk).

74. What is "groundedness" and how do you measure it?

Groundedness = degree to which the generated answer is anchored in retrieved context. Equivalent to faithfulness in most usages. Measurement: ask an LLM judge to label each statement in the answer as (a) supported by context, (b) contradicted by context, or (c) not in context (model brought outside knowledge). Groundedness score = fraction supported.

Some systems also flag (b) and (c) cases separately because they have different failure modes.

75. How do you handle subjective or open-ended questions in eval?

For questions without one right answer ("explain the trade-offs of microservices"), use rubric-based LLM-as-judge: define 3–5 criteria (completeness, accuracy, clarity, structure), have the judge score each on 1–5. Aggregate. Also: pairwise preference (which of these two answers is better?) is more reliable than absolute scoring for subjective output. Human evaluation on a sample is still the gold standard but expensive - use LLM judges for scale, human spot-checks for calibration.

76. What tools do you know for RAG evaluation?

RAGAS: open-source, faithfulness/relevance/precision/recall metrics, integrates with LangChain. **TruLens**: also open-source, focuses on app-level evals with feedback functions. **DeepEval**: pytest-style evals, easy to integrate in CI. **Arize Phoenix**: open-source observability + evals. **LangSmith**: hosted, by LangChain - strong on tracing and dataset-driven testing. **Langfuse**: open-source observability, good evaluation features. Most production systems combine one observability tool with custom eval scripts.

77. How do you evaluate citations?

Score on: (1) **Citation precision** - of the citations the system provides, what fraction support the claim they're attached to? (2) **Citation recall** - of the claims that *could* be cited, what fraction actually have citations? (3) **Citation correctness** - do citations point to the right source/chunk, not just any source? Use LLM-as-judge with the answer, the citations, and the source documents to check each. Citations are critical for legal, medical, and enterprise RAG - never ship without explicit citation evaluation.

78. How do you set up human-in-the-loop evaluation?

(1) **Sampling strategy**: don't review everything - sample interesting cases (low confidence, refused queries, user thumbs-down, high-stakes domains). (2) **Review UI**: side-by-side query, retrieved chunks, generated answer, expected answer; reviewers label issues by category. (3) **Calibration**: have multiple reviewers double-rate 10% to measure inter-rater agreement. (4) **Closing the loop**: failed cases become new golden set entries; recurring failure categories drive engineering work. Run a review cadence (e.g., 100 cases / week).

79. What's a "smoke test" for RAG and what should be in it?

A small set (10–20 queries) that you run on every deploy to catch obvious regressions before they hit production. Should include: (1) one query per major topic in your corpus; (2) one "I don't know" query (out of scope); (3) one adversarial query (prompt injection attempt); (4) one query with citation requirement; (5) one query with multi-hop reasoning. Smoke tests should run in under 60 seconds - not a full eval, but a fast canary.

80. How do you measure "user satisfaction" in RAG without explicit ratings?

Implicit signals: (1) **Engagement** - did the user click a citation, scroll the answer, expand details? (2) **Follow-up rate** - did the user immediately rephrase their question (sign the answer wasn't helpful)? (3) **Abandonment** - did they leave the session right after the answer? (4) **Copy events** - did they copy the answer (sign of value)? (5) **Escalation** - did they ask to talk to a human? Combine into a satisfaction proxy score, validate against a sample of explicit ratings.

Category 5 - Production & Advanced RAG (Questions 81–100)

81. How do you reduce hallucinations in a RAG system?

Defense in depth: (1) **Strong system prompt** - "Only answer from the provided context. If insufficient, say so." (2) **Force citations** - require the model to cite chunk IDs for every claim. (3) **Refusal training** - include refusal examples in few-shot. (4) **Output validation** - post-generation, check that every cited chunk exists and every uncited claim is also in context. (5) **Faithfulness eval in CI**. (6) **Calibrated temperature** - lower temperature (0–0.3) for factual RAG. (7) **Better retrieval** - most hallucinations are retrieval failures; the model invented an answer because the right context wasn't there.

82. How do you force the LLM to cite sources reliably?

(1) Assign each retrieved chunk a stable ID in the prompt (e.g., [SOURCE_1], [SOURCE_2]). (2) Use a system prompt requiring inline citations after every claim. (3) Provide few-shot examples of well-cited answers. (4) Use structured outputs (JSON schema with {answer, citations: [chunk_ids]}). (5) Post-process: parse citations, verify each cited chunk ID exists, optionally regenerate if citations are missing. (6) Add a verification step: ask a second LLM call "does each claim in this answer have a valid citation?" Combine these for >95% citation reliability.

83. What is HyDE (Hypothetical Document Embeddings)?

HyDE: instead of embedding the user's query and searching, first ask an LLM to *write a hypothetical answer*, then embed the hypothetical answer and search. Why it works: answers tend to be in the same vocabulary/style as the documents that contain them, so they retrieve better than questions. Trade-off: extra LLM call (latency + cost), and if the hypothetical answer is wrong-direction, retrieval suffers. Useful for short or vague queries; less needed when queries are already well-formed.

84. What is query rewriting and when do you need it?

Query rewriting transforms the user's raw query into a better retrieval query, using an LLM. Common transformations: (1) **Expand acronyms and abbreviations**; (2) **Add context from conversation history**; (3) **Clarify pronouns** ("the second one" → "the second product mentioned"); (4) **Decompose multi-part questions**; (5) **Generate alternative phrasings**

for multi-query retrieval. Needed when conversational follow-ups are common, when queries are short/vague, or when domain language differs from user language.

85. What is multi-query retrieval?

Generate multiple variations of the query (using an LLM), retrieve for each, and merge results (RRF). Why: a single query embedding might miss documents that match the *intent* but use different words. Trade-off: more LLM calls and more retrieval calls - cost goes up. Especially useful for ambiguous or vague queries. Standard pattern in production-grade RAG.

86. How do you handle multi-hop / multi-document questions?

Multi-hop = "Who is the CEO of the company that makes the iPhone?" requires retrieving info about Apple first, then about Tim Cook. Approaches: (1) **Query decomposition** - break into sub-questions, retrieve for each. (2) **Iterative retrieval** - retrieve, generate intermediate answer, use that to formulate a new retrieval, repeat. (3) **Self-RAG / Self-Query** - let the LLM decide when to retrieve again. (4) **Graph-based RAG** - use a knowledge graph alongside vector search. (1) and (2) are most common in production.

87. What is "self-RAG"?

Self-RAG is a pattern where the LLM decides at each step whether to retrieve again, whether the retrieved context is relevant, and whether its own answer is well-supported - using special control tokens during generation. More autonomous than fixed RAG pipelines. Trade-off: more LLM calls (cost, latency) and less predictable behavior, but better at multi-hop and "I don't know" handling. Implementations: Self-RAG paper (Asai et al.), CRAG, and various agent-based RAG patterns.

88. What is "agentic RAG"?

Agentic RAG treats retrieval as a tool the agent can call multiple times with different strategies - different queries, different indexes, different filters - based on what the agent learns about the question. Useful for: complex queries, multi-source retrieval, queries that span structured (SQL) and unstructured (vector) data. Trade-off: significantly higher cost and latency, and harder to evaluate. Use only when simpler RAG isn't enough.

89. What is "RAG fusion"?

A specific technique: generate multiple query variations → retrieve for each → fuse results using Reciprocal Rank Fusion → pass top results to LLM. Combines multi-query retrieval with RRF. Improves recall on vague or ambiguous queries while keeping precision. Implementation is straightforward and gives consistent improvements over single-query baseline.

90. How do you handle conversational RAG (follow-up questions)?

Two layers: (1) **Conversation memory** - keep recent turns in the prompt, but bounded (e.g., last 5 turns or 2000 tokens). (2) **Query rewriting using history** - before retrieving, ask an LLM to rewrite the latest query into a standalone form using conversation context. E.g.,

"What about for enterprise?" → "What pricing applies to enterprise customers?". Standalone queries embed much better than context-dependent fragments. Standard pattern in production chatbots.

91. How do you manage cost and latency in production RAG?

(1) **Caching**: exact-match cache for repeated queries; semantic cache for similar queries. (2) **Model routing**: cheap small model for easy queries, expensive large model for hard ones - classify with a fast first-pass. (3) **Tighter context**: rerank aggressively, pass fewer tokens. (4) **Async/streaming**: stream tokens to reduce perceived latency. (5) **Prompt compression** (LLMLingua, etc.) for long contexts. (6) **Embedding cache** at ingestion. (7) **Batch operations** wherever possible. (8) Track cost per query as a first-class metric.

92. What is semantic caching?

Cache that returns a cached response if a new query is *semantically similar* (cosine similarity above threshold) to a cached one - not just exact match. Implementation: embed every query, do a vector similarity check against cached query embeddings, return cached answer if similarity > 0.95 (or your threshold). Saves huge cost on common questions phrased many ways. Risks: returning stale cached answers when data has updated, and incorrect cache hits on similar-but-different questions - tune threshold carefully.

93. Design a production RAG system for a 10M-document corpus.

Ingestion: parallel pipeline (Spark/Ray), language-aware chunking, batch embedding via hosted API or self-hosted GPU, bulk insert into vector DB. Store metadata in Postgres for filtering. **Index**: HNSW on Qdrant or Weaviate; one collection per tenant if multi-tenant. **Retrieval**: hybrid (dense + BM25) with RRF merge, top-50; cross-encoder rerank to top-5; metadata pre-filtering. **Generation**: small fast model for simple Qs (Haiku/4o-mini), large model for hard ones; force-cite system prompt; output validation. **Observability**: Langfuse for traces; RAGAS metrics in CI; production dashboards for cost/latency/quality. **Eval**: 500-Q golden set, weekly review of 100 production samples.

94. How do you handle PII in a RAG pipeline?

(1) **At ingestion**: detect PII (Microsoft Presidio, AWS Comprehend, regex for known patterns) and either redact, pseudonymize, or quarantine. (2) **At query time**: detect PII in user queries, log carefully (or not at all). (3) **At storage**: encrypt at rest, restrict vector DB access. (4) **At generation**: post-process outputs to redact PII before returning. (5) **For compliance** (HIPAA, GDPR): document data flows, support deletion-on-request (which means tracking which embeddings came from which source doc - non-trivial in vector DBs).

95. How do you implement prompt injection defenses in RAG?

(1) **Trust boundary** - system prompt is trusted, retrieved context is untrusted (it might contain "ignore previous instructions" planted in documents). (2) **Sandwich pattern** - restate the system instructions *after* the retrieved context. (3) **Output validation** - check that the output addresses the user's actual question, not the planted instruction. (4) **Structured outputs** - JSON schemas make it harder for prompt injections to hijack the response. (5)

Detection - flag retrieved chunks containing suspicious phrases. (6) **Eval against injection attempts** as part of CI. Defense-in-depth, not a single fix.

96. How do you build RAG over structured data (databases)?

Don't use pure vector RAG - use **Text-to-SQL** patterns instead: the LLM generates SQL from the question, you execute it, return results. Optionally combine: vector retrieve documentation about the schema/business logic, then LLM generates SQL using that context. For hybrid structured + unstructured (a question that needs both), build a tool-using agent that can call both vector search and SQL - and have it decide which to use.

97. What is "long-context RAG" and how does it differ from regular RAG?

Long-context RAG retrieves more chunks (50-100 instead of 3-5) and relies on models with large context windows (Claude, Gemini, GPT-4 long context) to find relevant info themselves. Pros: simpler retrieval, robust to retrieval errors. Cons: cost scales with context, "lost in the middle" still happens, slower. Sweet spot: retrieve aggressively (high recall), then either rerank tight (traditional) or pass it all to a long-context model (long-context RAG). Hybrid is often best.

98. What is a "fact-checker" pattern in RAG?

After generating an answer, a second LLM call (the fact-checker) verifies each claim against the retrieved context. If any claim is unsupported, either: (a) regenerate the answer, (b) flag the response for human review, or (c) abstain. Trade-off: ~2x cost and latency, but dramatically improves faithfulness. Used in high-stakes domains (medical, legal, financial). Less needed for low-stakes use cases.

99. How do you version a RAG system?

Version everything: (1) **Prompts** - store in git, version with semver, tag in production logs. (2) **Models** - pin embedding and LLM model versions; don't auto-upgrade. (3) **Chunking strategy** - chunking params are part of the version; changes require re-indexing. (4) **Index** - xusvector DB collections can be versioned (`docs_v1`, `docs_v2`) for safe rollouts. (5) **Evals** - track which version a metric was measured on. Without versioning, you can't reproduce issues or do safe rollbacks.

100. What's the future of RAG (as of 2026)?

Trends to watch: (1) **Long-context models** are reducing the *need* for aggressive retrieval - many short-corpus use cases are moving to "just put it all in context." (2) **Multimodal RAG** - embedding and retrieving over images, audio, video. (3) **Agentic RAG** - retrieval becomes one tool among many. (4) **Personalized retrieval** - embeddings adapted per user/session. (5) **Better eval** - synthetic eval generation, more reliable LLM judges, in-context eval. (6) **End-to-end trained retrieval** - joint training of retriever and generator for specific domains. RAG isn't going away; it's getting more sophisticated.

How to Use This Doc

Week 1: Read all 100 Q's, mark which you can answer cold.

Week 2: Deep-study your weak categories. For every gap, build a small project.

Week 3: Mock interviews with a friend using this doc.

Week 4: Re-read just the answers - by now they should feel obvious.

If you can confidently answer 70+ out of 100, you're ready for senior RAG interviews. If you hit 90+, you're in the top 5%.

aryanwadhavkar